LAB211 Report

Software Developer Management

Student Name: Nguyễn Phạm Trí
Student Code: CE192008
Class: SE2047

---

1. Introduction

This report presents the development of the **Software Developer Management** system. The application is implemented as a **Java console program** that applies **Object-Oriented Programming (OOP)** principles and **Computational Thinking (CT)** to manage developers and their projects in an organized and reliable way.

The system supports typical management needs such as:

- Managing **Developer** data (add, search, update salary, remove, sort, filter by programming language).
- Managing **Project** data (add, list projects by developer, calculate total experience).
- Ensuring **data integrity** with validation rules (ID format, uniqueness, salary threshold, future start dates).
- Persisting data using **text files** (read at startup, write when saving).

The system is designed to be maintainable and extendable by separating responsibilities into clear modules: **View (UI)**, **Controller (business logic)**, **Model (data objects)**, **File I/O (persistence)**, and **Utils (validation and helpers)**.

---

2. Requirement Analysis

2.1 Core Entities

The project manages two main entities:

Developer

- devId: unique identifier (format DEVxxx)
- fullName: at least 2 words

- languages: list of programming languages (stored as [Java, C++] in file; may be empty [] if allowed)
- salaryUsd: integer, minimum 1000

Project

- projectId: unique identifier (your implementation requires uniqueness; no strict prefix constraint)
- devId: must reference an existing developer
- projectName: not empty
- durationMonths: integer, minimum 1
- startDate: must be a **future** date (format dd/MM/yyyy in file)

2.2 Main Functionalities (Menu-Based)

Your program implements these management functions:

1. List all Developers
2. Add a new Developer (supports continuous adding)
3. Search for a Developer by ID
4. Update a Developer's salary by ID (invalid input returns to menu immediately)
5. List all Developers by Language
6. Add a new Project (developer must exist)
7. List all Projects by Developer (grouped by each developer)
8. Calculate Total Experience by Dev ID (sum duration months)
9. Remove a Developer by ID (only if developer has no projects)
10. Sort Developers by Salary (ascending)
11. Save data to files
12. Quit program (with "save changes?" confirmation if modified)

2.3 Input Constraints and Validation

Key rules used to prevent broken data:

- Developer ID must match DEVxxx.
- Developer full name must contain at least two words.
- Salary must be ≥ 1000.
- Project duration must be ≥ 1 month.
- Project start date must be strictly after today.
- Project's Dev ID must exist in Developer list.
- IDs must be unique (Developer ID unique; Project ID unique).

---

3. Applying Computational Thinking

3.1 Decomposition

The system is decomposed into modules with single responsibilities:

1. **View Module (UI Layer)**

- Displays menus and tables.
- Collects and validates user inputs at UI-level (cannot be empty, must be integer, etc.).
- Provides two input modes:
    - **Loop input** (keep asking until valid): used for adding.
    - **One-shot input** (invalid → return to menu): used for update/remove/search.

2. **Controller Module (Business Logic Layer)**

- DeveloperController
    - Load/save developer data
    - Add/search/update salary/remove/filter/sort
- ProjectController
    - Load/save project data
    - Add project with cross-check to Developer list
    - List by dev, check if dev has projects, compute total experience

3. **File I/O Module (Persistence Layer)**

- Read and write plain text files:
    - developers file: includes list field [...]
    - projects file: comma-separated with date format
- Handles common issues such as whitespace and BOM \uFEFF.

4. **Utility Module**

- Validator: formats, constraints, parsing languages, parsing dates, future-date check, normalize IDs.
- FileUtils: read/write all lines, manage encoding, ignore blank lines.

3.2 Pattern Recognition

During design, several repeated patterns were identified and generalized:

- **Unique ID pattern**: both Developer and Project require unique IDs.
- **CRUD-like operations**: add/search/update/remove share the same input–validate–apply–message structure.
- **Validation pattern**: nearly every operation follows "input → validate format → validate existence/uniqueness → commit".
- **Persistence pattern**: load at startup; save on demand; prompt to save on exit if modified.
- **Cross-entity rule**: project references developer, so add/remove must check relationships.

3.3 Abstraction

The system abstracts the problem into essential data and behaviors:

- Developer abstraction focuses on:
  - identity (devId)
  - skill representation (languages)
  - compensation (salaryUsd)
  - utility: hasLanguage(), language formatting for UI/file
- Project abstraction focuses on:
  - identity (projectId)
  - ownership (devId)
  - timeline (durationMonths, startDate)
  - utility: date formatting for UI/file

Non-essential details (e.g., developer age, address, project budget) are intentionally excluded to keep the model clean and aligned with the assignment scope.

3.4 Algorithm Design

Below are key algorithms written in clear steps (the same style as the sample report).

A) Add New Developer (continuous mode)

1. Display "Add New Developer".
2. Input DevID; validate format DEVxxx; validate uniqueness.
3. Input Full Name; validate ≥ 2 words.
4. Input Languages; parse list; accept list (depending on rule: allow [] or require at least 1).
5. Input Salary; validate integer ≥ 1000.
6. Create Developer object and add into list.
7. Mark system state changed = true.
8. Show "ADD SUCCESS".
9. Ask user "Continue adding new? (Y/N)"; if Y, repeat; otherwise return to menu.

B) Update Developer Salary (one-shot ID + one-shot salary)

1. Display "Update Developer Salary".
2. Input DevID once:
   - If empty → show "Cannot be empty." → return to menu.
   - If invalid format → show "Invalid ID." → return to menu.
3. Check Dev exists:
   - If not exist → show "Developer ID does not exist!" → return menu.
4. Input Salary once:

    - If empty/invalid → show message → return menu.
    - If <1000 → show message → return menu.
5. Update salary in Developer object.
6. Mark changed = true.
7. Show "UPDATE SUCCESS".
8. Ask "Continue updating new? (Y/N)"; if Y repeat; else return menu.

C) Add New Project (continuous mode)

1. If developer list is empty → show warning and return.
2. Input ProjectID; validate non-empty and uniqueness.
3. Let user select DevID from list (prevents invalid DevID input).
4. Input Project Name (non-empty).
5. Input Duration months (integer ≥ 1).
6. Input Start Date (future date).
7. Validate devId exists (controller check).
8. Create Project object and add into project list.
9. Mark changed = true.
10. Show "ADD PROJECT SUCCESS".
11. Ask "Continue adding new project? (Y/N)"; repeat if Y.

D) Remove Developer (only when no projects)

1. Input DevID once (invalid → return menu).
2. Check Dev exists; if not → message → return.
3. Check if dev has any projects:
    - If yes → show "Cannot delete: Developer is assigned to projects." → return.
4. Remove developer from list.
5. Mark changed = true.
6. Show "REMOVE SUCCESS".
7. Ask user to continue removing; if no → return menu.

E) Save and Exit

- **Save**: write both lists (developers + projects) to files; if successful → reset changed=false.
- **Exit**:
    - If changed=false → exit immediately.
    - If changed=true → ask "Save changes? (Y/N)":
        - If Y and save success → exit.
        - If Y and save fails → return to menu.
        - If N → exit without saving.

---

4. System Design (UML)

4.1 Main Classes

- Developer (model)
- Project (model)
- DeveloperController
- ProjectController
- IFileReadWrite<T> (generic interface)
- DeveloperFileHelper implements IFileReadWrite<Developer>
- ProjectFileHelper implements IFileReadWrite<Project>
- Validator, FileUtils (utils)
- View, Menu (UI)
- Main (entry point)

4.2 Relationships (text-based UML description)

- **Main** “has” View, DeveloperController, ProjectController.
- **DeveloperController** “has” List<Developer> and uses DeveloperFileHelper for persistence.
- **ProjectController** “has” List<Project> and uses ProjectFileHelper for persistence.
- **ProjectController** depends on **DeveloperController** when adding project (to validate DevID existence).
- **View** uses **Validator** to validate and parse inputs (ID format, salary, date parsing).
- **FileHelper classes** use **FileUtils** to read/write files.

4.3 Dependencies

Main --> View

Main --> DeveloperController

Main --> ProjectController

DeveloperController --> "1" IFileReadWrite<Developer>

ProjectController --> "1" IFileReadWrite<Project>

DeveloperFileHelper ..> FileUtils

DeveloperFileHelper ..> Validator

ProjectFileHelper ..> FileUtils

ProjectFileHelper ..> Validator

View ..> Validator

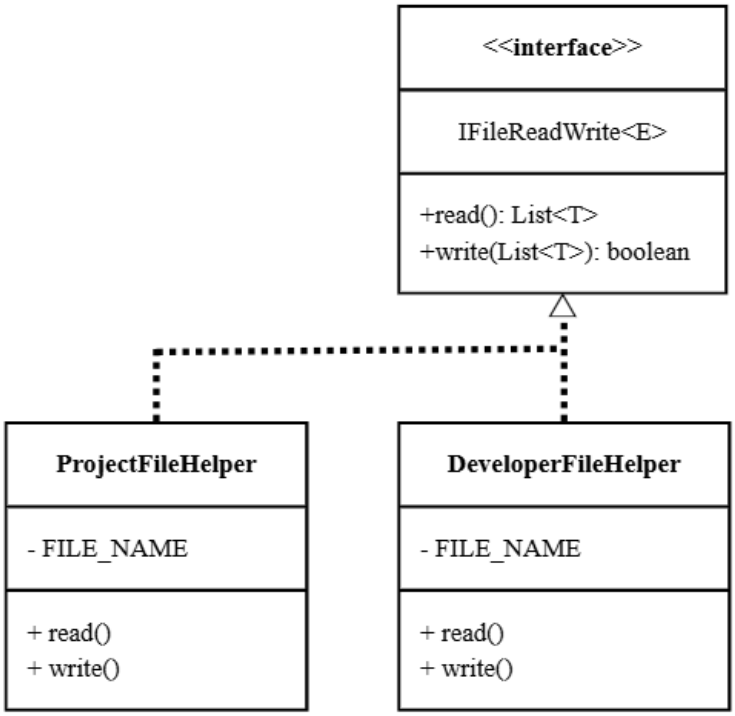
View ..> Developer

View ..> Project

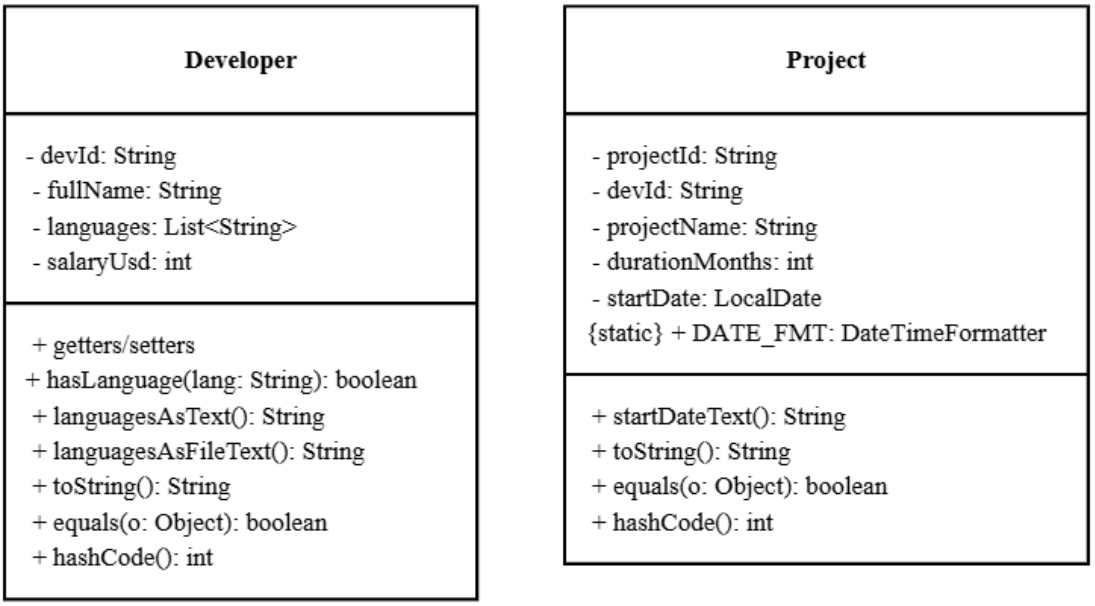
ProjectController ..> DeveloperController : validate DevID exists

Class Diagram

fileio layer



Model layer



controller layer

DeveloperController

- developers: List<Developer>
- fileHelper: IFileReadWrite<Developer>

+ load(): void
+ save(): void
+ getAll(): List<Developer>
+ existsId(devId: String): boolean
+ findById(devId: String): Developer
+ add(d: Developer): boolean
+ updateSalary(devId: String, newSalary: int): boolean
+ removeIfNoProject(devId: String, projectCtrl: ProjectController): boolean
+ filterByLanguage(lang: String): List<Developer>
+ sortBySalaryAsc(): void

ProjectController

- projects: List<Project>
- fileHelper: IFileReadWrite<Project>

+ load(): void
+ save(): void
+ getAll(): List<Project>
+ existsProjectId(projectId: String): boolean
+ add(p: Project, devCtrl: DeveloperController): boolean
+ getByDevId(devId: String): List<Project>
+ hasProjectsOfDev(devId: String): boolean
+ totalExperienceMonths(devId: String): int

utils layer

FileUtils

+ readAllLines()
+ writeAllLines()

Validator

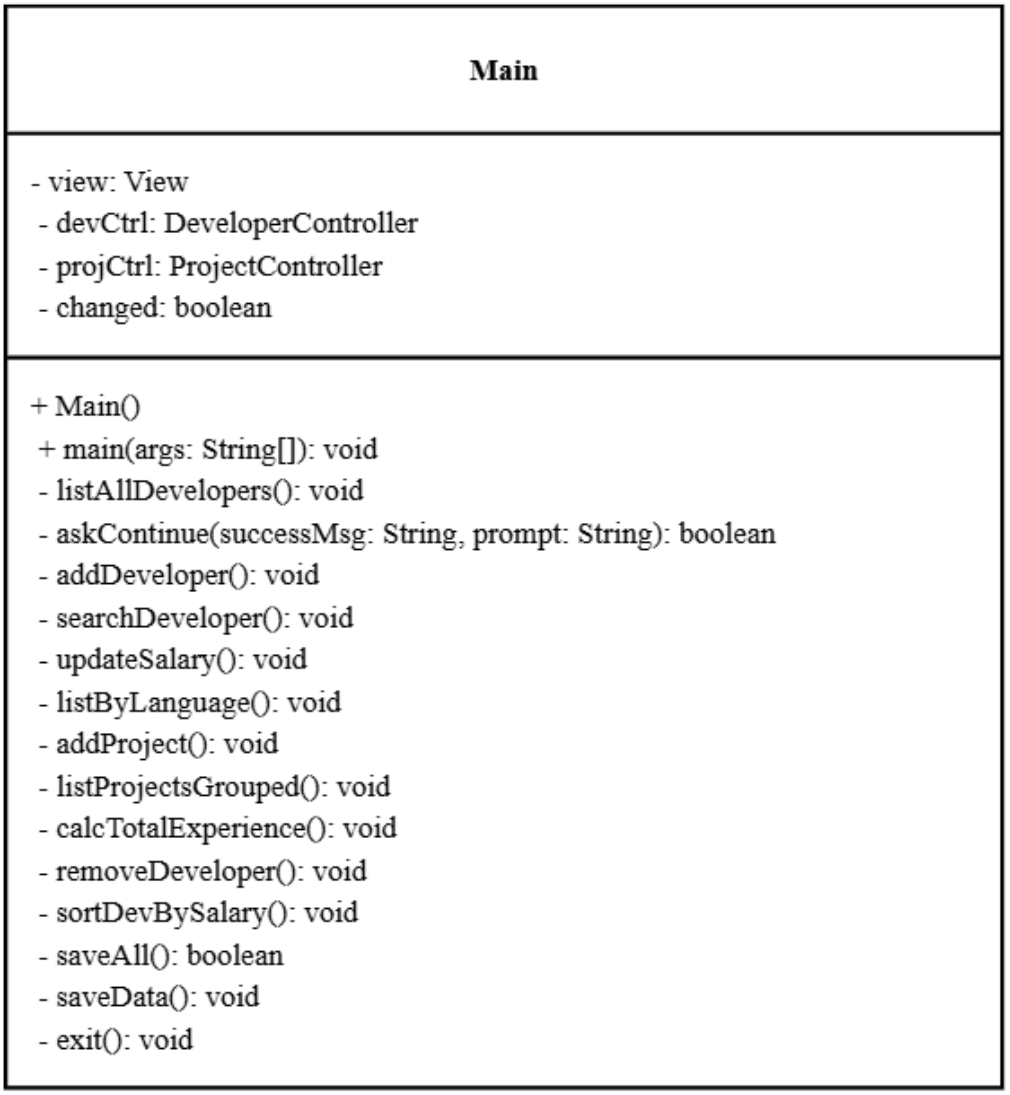
- DEV_ID: Pattern
- DATE_FMT: DateTimeFormatter

+ validDevId(id: String): boolean
+ validFullName(name: String): boolean
+ parseLanguages(raw: String): List<String>
+ validLanguages(langs: List<String>): boolean
+ validSalary(salary: int): boolean
+ validProjectName(name: String): boolean
+ validDuration(months: int): boolean
+ parseDate(s: String): LocalDate
+ isFutureDate(d: LocalDate): boolean
+ normalizeId(id: String): String

view layer

Menu
+showMenu()

View
– sc: Scanner
+ showMessage(msg: String): void + showMainMenu(): int + showDevelopersTable(list: List<Developer>): void + showDeveloperDetail(d: Developer): void + showProjectsOfDeveloper(d: Developer, projects: List<Project>): void + readOptional(prompt: String): String + inputNonEmpty(label: String): String + inputNonEmptyOnce(label: String): String + inputDevIdAnyOnce(): String + inputDevIdNew(existing: List<Developer>): String + inputFullName(): String + inputLanguages(): List<String> + inputSalary(): int + inputSalaryOnce(): Integer? + inputProjectIdNew(existing: List<Project>): String + inputProjectName(): String + inputDurationMonths(): int + inputFutureDate(): LocalDate + chooseDevIdFromList(devs: List<Developer>): String + parseIntSafe(s: String, fallback: int): int



---

5. Flowchart

PSEUDOCODE

START

Main():

devCtrl.load()

projCtrl.load()

changed = false

```
  LOOP forever:
    choice = view.showMainMenu()

    SWITCH choice:
      1: listAllDevelopers()
      2: addDeveloperLoop()
      3: searchDeveloperOnce()
      4: updateSalaryLoop()
      5: listByLanguage()
      6: addProjectLoop()
      7: listProjectsGrouped()
      8: calcTotalExperience()
      9: removeDeveloperLoop()
      10: sortBySalaryAscAndDisplay()
      11: saveData()
      12: exitWithSavePrompt()
      default: print "This function is not available."
END


FUNCTION listAllDevelopers():
  print header
  view.showDevelopersTable(devCtrl.getAll())
```

```
FUNCTION askContinue(successMsg, prompt) -> boolean:
  print successMsg
  ans = view.readOptional(prompt)
  return (ans == "Y" ignoring case)


FUNCTION addDeveloperLoop():
  REPEAT:
    id = view.inputDevIdNew(devCtrl.getAll())
    name = view.inputFullName()
    langs = view.inputLanguages()
    salary = view.inputSalary()

    ok = devCtrl.add(new Developer(id, name, langs, salary))
    IF ok == false:
      print "Add failed"
      RETURN

    changed = true
  UNTIL askContinue("=== ADD SUCCESS ===", "Continue adding new? (Y/N): ") ==
false


FUNCTION searchDeveloperOnce():
```

```
  id = view.inputDevIdAnyOnce()
  IF id == null: RETURN
  d = devCtrl.findById(id)
  IF d == null: print "Developer ID does not exist!" ; RETURN
  view.showDeveloperDetail(d)


FUNCTION updateSalaryLoop():   // one-shot invalid -> menu
  REPEAT:
    id = view.inputDevIdAnyOnce()
    IF id == null: RETURN
    IF devCtrl.findById(id) == null:
      print "Developer ID does not exist!"
      RETURN

    newSalary = view.inputSalaryOnce()
    IF newSalary == null: RETURN

    ok = devCtrl.updateSalary(id, newSalary)
    IF ok == false:
      print "Update failed"
      RETURN

    changed = true
```

```
  UNTIL askContinue("=== UPDATE SUCCESS ===", "Continue updating new? (Y/N): ")
== false



FUNCTION listByLanguage():

  lang = view.inputNonEmpty("Language: ")

  list = devCtrl.filterByLanguage(lang)

  IF list empty: print "No developer matches!" ; RETURN

  view.showDevelopersTable(list)



FUNCTION addProjectLoop():

  IF devCtrl.getAll() empty:

    print "No developers available. Please add a developer first."

    RETURN


  REPEAT:

    projId = view.inputProjectIdNew(projCtrl.getAll())

    devId = view.chooseDevIdFromList(devCtrl.getAll())

    IF devId == null: print "No developer selected!" ; RETURN


    name = view.inputProjectName()

    duration = view.inputDurationMonths()

    startDate = view.inputFutureDate()
```

```
    ok = projCtrl.add(new Project(projId, devId, name, duration, startDate), devCtrl)
    IF ok == false:
      print "Add project failed"
      RETURN

    changed = true
  UNTIL askContinue("=== ADD PROJECT SUCCESS ===", "Continue adding new
project? (Y/N): ") == false


FUNCTION listProjectsGrouped():
  devs = devCtrl.getAll()
  IF devs empty: print "(No developers)" ; RETURN
  FOR each dev in devs:
    ps = projCtrl.getByDevId(dev.devId)
    view.showProjectsOfDeveloper(dev, ps)


FUNCTION calcTotalExperience():
  id = view.inputDevIdAnyOnce()
  IF id == null: RETURN
  IF devCtrl.findById(id) == null:
    print "Developer ID does not exist!"
    RETURN
  total = projCtrl.totalExperienceMonths(id)
```

```
  print total



FUNCTION removeDeveloperLoop():
  REPEAT:
    id = view.inputDevIdAnyOnce()
    IF id == null: RETURN
    IF devCtrl.findById(id) == null:
      print "Developer ID does not exist!"
      RETURN
    IF projCtrl.hasProjectsOfDev(id):
      print "Cannot delete: Developer is assigned to projects."
      RETURN

    ok = devCtrl.removeIfNoProject(id, projCtrl)
    IF ok == false:
      print "Remove failed"
      RETURN

    changed = true
  UNTIL askContinue("=== REMOVE SUCCESS ===", "Continue removing new? (Y/N): ")
== false



FUNCTION sortBySalaryAscAndDisplay():
```

```
  devCtrl.sortBySalaryAsc()
  view.showDevelopersTable(devCtrl.getAll())



FUNCTION saveAll() -> boolean:
  TRY:
    devCtrl.save()
    projCtrl.save()
    print "Saved to files successfully."
    RETURN true
  CATCH:
    print "Save failed"
    RETURN false



FUNCTION saveData():
  IF changed == false:
    print "No changes to save."
    RETURN
  IF saveAll() == true:
    changed = false



FUNCTION exitWithSavePrompt():
  IF changed == false:
```

```
  print "Bye!"
  EXIT
ans = view.readOptional("Save changes? (Y/N): ")
IF ans == "Y" ignoring case:
  IF saveAll() == false:
    print "Save failed. Return to menu."
    RETURN
print "Exiting..."
EXIT
```

---

6. Implementation & Testing

6.1 Implementation Notes

- Java console application with clear package separation.
- Data persistence uses plain text files:
  - Developers line contains list in brackets, requiring careful parsing.
- Robust input handling:
  - “Loop until valid” for add operations.
  - “One-shot return to menu” for update/remove/search (matches the usability requirement).
- Defensive handling:
  - Ignore blank lines during file read.
  - Strip BOM (\uFEFF) to prevent hidden-character ID bugs.
  - Normalize IDs to uppercase for consistent matching.

6.2 Representative Test Cases

1. **Add Developer (valid)**
- Input: DEV001, “Nguyen Van A”, Java, C++, 1500
- Expected: added successfully; appears in list.
2. **Add Developer (duplicate ID)**
- Add DEV001 again
- Expected: rejected with “Duplicated ID”.

3. **Search Developer (non-existent)**
- Input: DEV999
- Expected: “Developer ID does not exist!”
4. **Update salary (invalid salary)**
- Input: DEV001, salary 900
- Expected: “Invalid salary” and return to menu (no loop).
5. **Add Project (dev doesn’t exist)**
- Attempt to assign DevID not in list
- Expected: add fails (controller blocks).
6. **Remove Developer with existing projects**
- Dev has at least one project
- Expected: reject removal with message.
7. **Save and Reload**
- After modifications, save.
- Restart program.
- Expected: loaded data matches saved state.

---

7. Appendix

7.1 Source Files (suggested listing)

- Main.java
- model/Developer.java, model/Project.java
- controller/DeveloperController.java, controller/ProjectController.java
- fileio/IFileReadWrite.java, fileio/DeveloperFileHelper.java, fileio/ProjectFileHelper.java
- utils/Validator.java, utils/FileUtils.java
- view/View.java, view/Menu.java

7.2 Sample Data Format

developers.txt

- DEV001, Nguyen Van A, [Java, C++], 5000

projects.txt

- P01, DEV001, E-Commerce Platform, 12, 01/01/2027

7.3 How to Run

1. Place developers.txt and projects.txt in the expected folder path.
2. Run Main.

3. Use menu functions; save before exit if required.